

Arrays in JS

Lec-5

Arrays in JS

- An Array is an **object type** designed for **storing data collections**.
- An array can **hold many values under a single name**, and you can **access the values by referring to an index number**.
- Key **Characteristics of an Array** are as follows:
 1. **Zero-Indexed**: The **first element** is always at **index 0**, the **second at 1**, and so on.
 2. **Ordered Collection**: Elements are stored in a specific sequence, and that order is maintained based on their index.
 3. **Dynamic and Resizable**: Unlike some other languages, **JavaScript arrays do not have a fixed size**; they grow or shrink automatically as you add or remove elements.
 4. **Heterogeneous (Mixed Types)**: A single array can hold elements of different data types simultaneously, such as a mix of strings, numbers, objects, and even other arrays.
 5. **Reference Type**: Arrays are technically **objects**. The **typeof** operator returns **object** for an array, and they are copied by reference rather than by value.
- It is always an best practice to declare an Array using **const** keyword.
- Generally there are two ways of declaring an array:

Way-1 : Using Array literal.

Way-2 : Using 'new' keyword.

Using Array literal	Using 'new' keyword
<pre>const cars = ['Volvo','BMW','Audi','Hyndai']</pre>	<pre>const cars2 = new Array('Volvo','BMW','Audi','Hyndai')</pre>

- In most cases we always prefer **declaring an array by creating an Array literal (Way-1)**.
- We can **access an array element** by referring to the **index number**.
- **Eg :**

<pre>const sports = ['football','cricket','basketball','golf','rugby'] console.log(sports[3]); // golf console.log(sports[1]); // cricket</pre>	
---	--

- We can also create an empty array, and provide elements later.
- **Eg :**

<pre>const groceries = [] groceries[0] = "Egg" groceries[1] = "Milk" groceries[2] = "Sugar" groceries[3] = "bread" console.log(groceries); //['Egg', 'Milk', 'Sugar', 'bread']</pre>	
---	--

- We can any existing element of an existing array in the following way:

<pre>const stationary = ['Pencil','Pen','Eraser','Exam Pad'] console.log(stationary); //['Pencil', 'Pen', 'Eraser', 'Exam Pad'] stationary[2] = 'Sharpner' console.log(stationary); // ['Pencil', 'Pen', 'Sharpner', 'Exam Pad']</pre>	
---	--

- We can find length of any array using the **.length** property.

<pre>const kitchenEquip = ['knife','spatula','stirrer','mixer'] console.log(kitchenEquip.length); // 4</pre>	
--	--

Array Methods

- **.push()** : Adds a new element **at the end** of an array.

```
let fruits = ['kiwi','strawberry','watermelon']
console.log(fruits);
// output: kiwi, strawberry, watermelon
fruits.push('muskmelon')
console.log(fruits);
// output: kiwi, strawberry, watermelon, muskmelon
```

What if we merge two arrays into single array by using .push() method ?

```
const americanCars = ['Ford','Tesla','GM']
const germanCars = ['Volkswagon','BMV','Audi']

americanCars.push(germanCars)
console.log(americanCars);
// output: ['Ford', 'Tesla', 'GM', Array(3)]
```

If we observe above example carefully we can see that, .push() methods considers the 2nd array as one single element and adds up with the elements of 1st array. To overcome this we can use **.concat()** or **spread operator**.

- **.concat()** : Creates a new array by merging (concatenating) the existing arrays.

```
const americanCars = ['Ford','Tesla','GM']
const germanCars = ['Volkswagon','BMV','Audi']

const mergedArrays = americanCars.concat(germanCars)
console.log(mergedArrays);
// output: ['Ford', 'Tesla', 'GM', 'Volkswagon', 'BMV', 'Audi']
```

Alternatively, we can also use **Spread Operator**.

Spread Operator (...)

- This operator splits iterables into individual elements.
- **Eg-1 :**

```
let string1 = 'Enginner'
console.log(...string1); // output: E n g i n e e r
```

- **Eg-2 :**

```
const americanCars = ['Ford','Tesla','GM']
const germanCars = ['Volkswagon','BMV','Audi']

const mergedArrays = [...americanCars,...germanCars]
console.log(mergedArrays);
// output: ['Ford', 'Tesla', 'GM', 'Volkswagon', 'BMV', 'Audi']
```

- **Eg-3 :**

```
const domesticAnimals = ['dog','cat']
const farmAnimals = ['cow','pig','chickens','goat']

const animals = ['lion','zebra', ...domesticAnimals, 'cheetah','panther', ...farmAnimals]
console.log(animals);
// output: ['lion', 'zebra', 'dog', 'cat', 'cheetah', 'panther', 'cow', 'pig', 'chickens', 'goat']
```

- **pop()** : This method **removes the last element** from an array.

```
const animals = ['lion','zebra', ...domesticAnimals, 'cheetah','panther', ...farmAnimals]
console.log(animals);
// output: ['lion', 'zebra', 'dog', 'cat', 'cheetah', 'panther', 'cow', 'pig', 'chickens', 'goat']
animals.pop()
console.log(animals);
// output: ['lion', 'zebra', 'dog', 'cat', 'cheetah', 'panther', 'cow', 'pig', 'chickens']
```

Array Methods

- **unshift()** : Adds one or more elements to the **beginning** of an array and returns the new length.

```
let cars = ['BMW', 'Audi'];  
cars.unshift('Ford');  
// ['Ford', 'BMW', 'Audi']
```

- **shift()** : Removes the **first** element from an array and returns that element.

```
let pets = ['Cat', 'Dog', 'Bird'];  
let first = pets.shift(); // 'Cat'  
// pets is now ['Dog', 'Bird']
```

- **includes()** : This method is a straightforward way to **check if an array contains a specific element**. It returns a **boolean result**. It is an **case sensitive** method.

```
const animals = ['Lion', 'Tiger', 'Bear'];  
console.log(animals.includes('Lion')); // true
```

Array Methods

- **indexOf()** : Returns the **first index** at which a given element can be found; returns **-1** if it is not present.

```
let tools = ['Hammer', 'Screwdriver', 'Hammer'];  
console.log(tools.indexOf('Hammer')); // 0
```

- **join()** :Concatenates all elements of an array into a **string**, separated by a specified string (comma by default).

```
let letters = ['J', 'S'];  
console.log(letters.join('-')); // "J-S"
```

- **slice()** :Returns a **shallow copy** of a portion of an array. It **does not change** the original array..

```
let nums = [10, 20, 30, 40];  
let middle = nums.slice(1, 3); // [20, 30] (index 1 to 2)
```

- **splice()** : **Changes** the contents of an array by removing or replacing existing elements and/or adding new ones

```
let colors = ['Red', 'Green', 'Blue'];  
colors.splice(1, 1, 'Yellow'); // At index 1, remove 1, add 'Yellow'  
// colors is now ['Red', 'Yellow', 'Blue']
```

Array Methods

- **flat()** : Creates a **new array with all sub-array elements concatenated into it** recursively up to the specified depth

```
let nested = [1, [2, [3]]];  
console.log(nested.flat(1)); // [1, 2, [3]]  
console.log(nested.flat(2)); // [1, 2, 3]
```

- **Array.from()** : Creates a **new array from an array-like or iterable object** (like a String or a Set).

```
console.log(Array.from('Hey')); // ['H', 'e', 'y']
```

- **Array.of()** : Creates a **new array with a variable number of arguments, regardless of the number or type of arguments.**

```
console.log(Array.of(7)); // [7] (Note: Array(7) creates an empty array with length 7)
```